

An aerial photograph showing a dense forest of trees with green and brown foliage. A narrow strip of sandy beach separates the forest from a calm body of water. The water reflects the surrounding greenery and sky. The overall tone is natural and serene.

# Sztuczna Inteligencja

w pracy Developera

Koncepcje, narzędzia i metody pracy



# AI dla developerów

Od zgłoszenia do sprawdzonej zmiany

- Dostarczyć AI wymaganie, właściwe źródła i zasady projektu
- Wprowadzić małą zmianę i sprawdzić zachowanie oraz diff
- Utrwalić sprawdzoną metodę jako prompt, instrukcję, Skill lub kontrakt agenta



# Mateusz Kulesza

Architekt rozwiązań, programista, konsultant i trener

- Fullstack Developer, Team Leader
- Solution Architect, Technical Architect
- Projektowanie architektury systemów
- Doradztwo, konsulting i szkolenia



# Uczestnik

Praca i oczekiwania

- Jakie zadania developerskie pojawiają się najczęściej?
- Jakie technologie i rodzaje projektów dominują w codziennej pracy?
- Jak wygląda obecne wykorzystanie AI?
- Które zadanie ma przebiegać sprawniej po szkoleniu?



# Jak pracujemy

- Krótkie wprowadzenia, praktyka i wspólne omówienia
- Pytania na bieżąco, dłuższe tematy do osobnej rozmowy
- Porównanie prób, także z poprawną pierwszą odpowiedzią
- Część podstawowa, rozszerzenia i gotowe punkty startowe
- Własne kopie projektu, kontrola źródeł i wyników przed akceptacją

3 dni · 09:00–15:00

Obiad około 12:00. Tempo dopasowane do grupy, materiały udostępniane etapami.

# Plan szkolenia

| Dzień | Kierunek pracy                     | Tematy                                                                                     |
|-------|------------------------------------|--------------------------------------------------------------------------------------------|
| 1     | Poznanie projektu i mała zmiana    | Koncepty AI, kontekst, przepływ przez kod, wymaganie i plan, implementacja                 |
| 2     | Sprawdzenie i poprawa zmiany       | Testy i TDD, debugging, code review, refactoring                                           |
| 3     | Ponowne użycie i przekazanie pracy | Prompty i Skills, delegowanie agentom, MCP, zarządzanie kontekstem, dokumentacja i handoff |

Decyzje człowieka i dowody poprawności towarzyszą pracy przez wszystkie dni.

# Koncepty AI

Model, kontekst i działanie w edytorze

# Model i aplikacja

| Pojęcie     | Definicja                                                | Przykład                                |
|-------------|----------------------------------------------------------|-----------------------------------------|
| Model / LLM | Model językowy generujący odpowiedź na podstawie wejścia | Model wybrany do rozmowy                |
| Aplikacja   | Program łączący model z interfejsem i środowiskiem pracy | VS Code z Copilotem                     |
| Harness     | Mechanizm organizujący kontekst i obsługę działań modelu | Udostępnienie pliku, wykonanie operacji |
| Kontekst    | Informacje dostępne modelowi przy danym wywołaniu        | Polecenie i fragment dokumentacji       |

# Czat i agent

| Pojęcie   | Definicja                                 | Przykład                                 |
|-----------|-------------------------------------------|------------------------------------------|
| Czat      | Interfejs wymiany wiadomości              | Panel rozmowy w edytorze                 |
| Sesja     | Stan rozmowy i prowadzonej pracy          | Rozmowa o koncepcji funkcji              |
| Asystent  | System pomagający przygotować odpowiedź   | Wyjaśnienie zachowania API               |
| Agent     | System prowadzący pętlę decyzji i działań | Odczyt dokumentacji i ocena wyniku       |
| Tryb      | Konfiguracja sposobu pracy                | Ask, Plan, Agent                         |
| Narzędzie | Funkcja odczytu lub działania             | Wyszukanie tekstu, uruchomienie kontroli |

# Generowanie odpowiedzi

Model tworzy tekst, wybierając prawdopodobną kontynuację dostępnego wejścia.

## Outline funkcji

Sensowna propozycja może powstać bez znajomości istniejącej implementacji.

## Rola w poleceniu

„Architekt rozwiązania” ukierunkowuje odpowiedź. Fakty o projekcie pochodzą z udostępnionych źródeł.

# Niedeterminizm

Powtórzenie tego samego polecenia może dać podobną lub różną odpowiedź.

## Zmienność

Styl, kolejność propozycji i sposób rozbicia funkcji mogą się różnić.

## Podstawa oceny

Wymaganie, źródła i kryteria oczekiwanego rezultatu obowiązują w każdej próbie.

# Porównywalne próby

| Warunek          | Co wpływa na porównanie                          |
|------------------|--------------------------------------------------|
| Model            | Nazwa, wersja i dostępna konfiguracja            |
| Polecenie        | To samo zadanie i sformułowanie                  |
| Historia         | Nowa rozmowa albo te same wcześniejsze ustalenia |
| Instrukcje       | Reguły projektu i aplikacji                      |
| Kontekst edytora | Dołączone pliki i zaznaczenie                    |
| Narzędzia        | Dostępne operacje i zwrócone informacje          |

Zmiana kilku warunków jednocześnie utrudnia wskazanie przyczyny różnicy.

# Historia rozmowy

Wcześniejsze wiadomości i odpowiedzi udostępnione modelowi w bieżącej sesji.

## Bieżąca rozmowa

Ustalony format odpowiedzi może wpływać na kolejne wiadomości.

## Nowa rozmowa

Ma inną historię. Aplikacja może ponownie dołączyć instrukcje projektu lub inne trwałe źródła.

# Kontekst a trening

| Mechanizm        | Co zawiera lub zmienia                                 | Przykład                            |
|------------------|--------------------------------------------------------|-------------------------------------|
| Kontekst sesyjny | Informacje w bieżącej rozmowie                         | Ustalony format outline'u           |
| Trwałe źródła    | Informacje udostępniane ponownie                       | Instrukcje projektu                 |
| Pamięć aplikacji | Przechowane informacje, jeśli funkcja działa           | Przywołane wcześniejsze ustalenie   |
| Trening modelu   | Parametry, czyli wartości wyuczone w procesie treningu | Przygotowanie modelu przez dostawcę |

Użycie reguły w odpowiedzi potwierdza dostęp do informacji. Samo nie dowodzi zmiany parametrów modelu.

# Polecenia i instrukcje

| Źródło tekstu        | Funkcja                                    | Przykład                          |
|----------------------|--------------------------------------------|-----------------------------------|
| Prompt               | Określa bieżące zadanie                    | Outline powiadomień o dostępności |
| Instrukcje projektu  | Opisują zasady powtarzalnej pracy          | Obowiązujący format opisu zmiany  |
| Instrukcje aplikacji | Określają zachowanie systemu               | Zasady korzystania z narzędzi     |
| Materiał do analizy  | Dostarcza informacji o przedmiocie zadania | Dokumentacja API, fragment kodu   |

Treść analizowanego dokumentu ma rolę źródła. Polecenie określa sposób jego wykorzystania.

# Polecenie developerskie

Cel: przygotuj outline powiadomień  
o dostępności produktu.

Kontekst: etap koncepcji, brak informacji  
o istniejącej implementacji.

Ograniczenia: bez kodu i wyboru biblioteki.

Format: zachowanie, pytania, założenia,  
sposób sprawdzenia.

Kryterium: każda propozycja ma opis  
oczekiwanego zachowania.

Braki: oddziel brakujące decyzje  
od przyjętych założeń.

# Zero-, one-, few-shot

| Wariant   | Liczba przykładów | Zastosowanie                            |
|-----------|-------------------|-----------------------------------------|
| Zero-shot | Bez przykładów    | Sam opis oczekiwanego rezultatu         |
| One-shot  | Jeden przykład    | Wzór układu odpowiedzi                  |
| Few-shot  | Kilka przykładów  | Różne przypadki oczekiwanego zachowania |

Liczona jest liczba przykładów zadania, niezależnie od liczby wiadomości.

Wzór układu używa pól `<status>` i `<uzasadnienie>`. Fakty o projekcie wymagają własnych źródeł.

# Fakty i założenia

| Status    | Podstawa                                 | Konsekwencja                                       |
|-----------|------------------------------------------|----------------------------------------------------|
| Fakt      | Odpowiednie źródło potwierdza informację | Opis zachowania z odniesieniem do kodu             |
| Założenie | Reguła przyjęta na potrzeby propozycji   | Wymaganie logowania czeka na decyzję produktu      |
| Brak      | Informacja pozostaje nieustalona         | Nieznany kanał powiadomienia wymaga doprecyzowania |

# Halucynacja

Nieprawdziwa lub wymyślona informacja wygenerowana i przedstawiona jako fakt.

## Przykład

Odpowiedź opisuje funkcję API, której nie ma we wskazanej wersji biblioteki.

## Status twierdzenia

Brak źródła pozostawia informację niepotwierdzoną.  
Kontrola rozstrzyga jej prawdziwość.

# Granice wiedzy

Wiedza z treningu może być niepełna lub nieaktualna.

## Cutoff

Granica danych treningowych. Nie gwarantuje znajomości wszystkich wcześniejszych faktów.

## Wersja biblioteki

Dokumentacja właściwej wersji dostarcza podstawy dla konkretnego API. Pomyłka modelu nie mierzy jego cutoff.

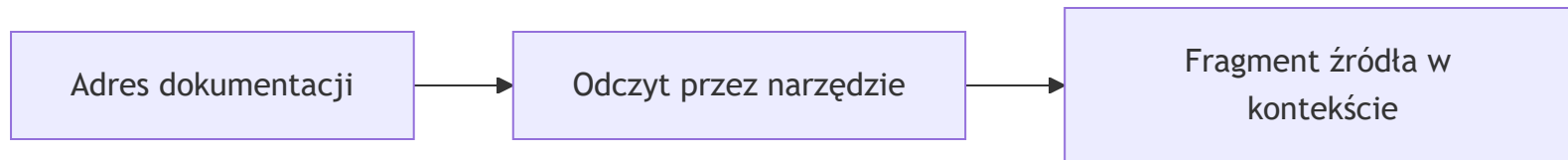
# Źródło i wersja

| Element     | Podstawa weryfikacji                                              |
|-------------|-------------------------------------------------------------------|
| Biblioteka  | Nazwa pakietu, którego dotyczy twierdzenie                        |
| Wersja      | Wersja rzeczywiście użyta, potwierdzona w instalacji lub lockfile |
| Twierdzenie | Konkretna funkcja, opcja albo zachowanie                          |
| Źródło      | Odczytany fragment dokumentacji tej wersji                        |
| Zakres      | Część twierdzenia potwierdzona przez fragment                     |

Link w odpowiedzi wskazuje możliwe źródło. Dowód odczytu wynika z wykonanej operacji.

# Narzędzie

Funkcja wykonująca operację i zwracająca wynik do dalszej pracy.



Wygenerowanie kodu i uruchomienie kodu to osobne zdarzenia.

# Dostęp, wykonanie, poprawność

| Poziom     | Pytanie kontrolne                           | Dowód                                        |
|------------|---------------------------------------------|----------------------------------------------|
| Dostęp     | Czy funkcja jest dostępna dla tego zadania? | Narzędzie i właściwy zasób                   |
| Wykonanie  | Czy konkretna operacja się odbyła?          | Wywołanie, parametry i wynik                 |
| Poprawność | Czy rezultat spełnia wymaganie?             | Właściwy zakres, definicja i kontrola wyniku |

Tekst „sprawdziłem” wymaga powiązania ze śladem wykonania.

# Token

Jednostka reprezentacji tekstu używana przez model.

## Podział tekstu

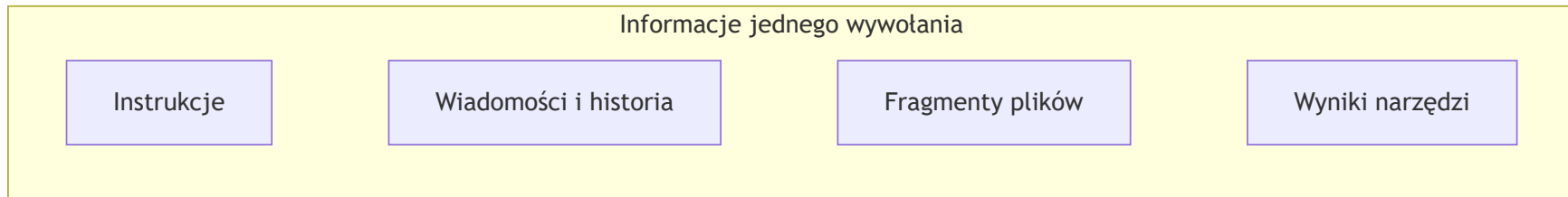
Słowo, nazwa symbolu i liczba mogą zajmować różną liczbę tokenów. Podział zależy od modelu.

## Limity i użycie

Miary zależą od usługi i konta. Obserwacja może obejmować czas, liczbę wywołań i dostępne zużycie.

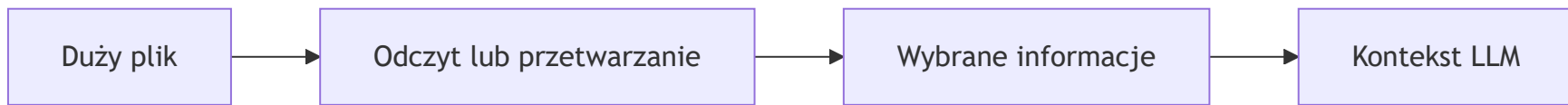
# Okno kontekstu

Ograniczona ilość informacji, którą model może przetworzyć w jednym wywołaniu.



Skracanie lub streszczanie historii może pominąć szczegóły. Limity generowanej odpowiedzi zależą od modelu i usługi.

# Załącznik i odczyt



Załączenie pliku nie dowodzi przekazania całej treści do modelu.

Narzędzie może przetwarzać zbiór poza oknem. Zakres operacji oraz zwrócone informacje wynikają z rzeczywistego przebiegu.

# Obliczenia i interpretacja

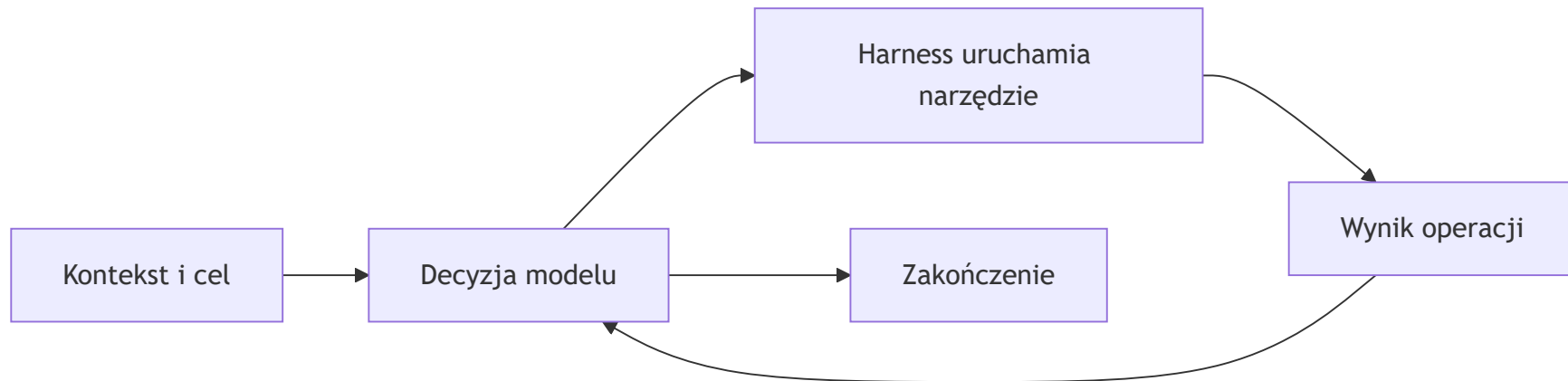
| Czynność                 | Wykonanie                                     | Podstawa kontroli                                    |
|--------------------------|-----------------------------------------------|------------------------------------------------------|
| Sumy, filtry, grupowanie | Narzędzie realizuje jawnie określoną operację | Definicja, pełny właściwy zakres, sprawdzony wycinek |
| Przygotowanie procedury  | LLM proponuje kod lub sposób obliczenia       | Reguła i założenia przed wykonaniem                  |
| Interpretacja wyniku     | LLM korzysta ze sprawdzonych rezultatów       | Źródła wniosków i granice wyjaśnienia                |

# Próbka i całość

| Cel                      | Potrzebny zakres                         | Kontrola                                     |
|--------------------------|------------------------------------------|----------------------------------------------|
| Rozpoznanie struktury    | Próbka i informacje o pliku              | Znaczenie pól, typy i jednostki              |
| Dokładna suma            | Pełny zbiór spełniający ustalone warunki | Zakres dat, rekordy przetworzone i odrzucone |
| Opis wybranego fragmentu | Jawnie wybrany wycinek                   | Wniosek ograniczony do tego wycinka          |

# Pętla agenta

Kolejne decyzje korzystają z wyników rzeczywiście wykonanych operacji.



Przykład: odczyt dokumentacji, ocena przydatności fragmentu, odpowiedź albo kolejny odczyt.

# Ask, Plan, Agent

| Tryb  | Cel                                   | Obserwowalny rezultat             |
|-------|---------------------------------------|-----------------------------------|
| Ask   | Wyjaśnienie lub propozycja            | Odpowiedź na pytanie albo outline |
| Plan  | Rozpoznanie i przygotowanie podejścia | Pytania, źródła i plan do oceny   |
| Agent | Wykonanie pracy z narzędziami         | Operacje i wyniki kontroli        |

Plan może korzystać z narzędzi rozpoznania. Agent może planować. Zmiana trybu nie musi zmieniać modelu.

# Granice działania

| Sytuacja           | Zakres                        | Kontrola człowieka           |
|--------------------|-------------------------------|------------------------------|
| Rozpoznanie        | Potrzebne źródła i odczyt     | Podstawa ustaleń             |
| Lokalna zmiana     | Uzgodniony fragment projektu  | Diff i oczekiwane zachowanie |
| Wykonanie kontroli | Właściwa operacja             | Wynik i zakres dowodu        |
| Publikacja         | Skutek w systemie zewnętrznym | Decyzja o publikacji         |

Dostęp wynika z konfiguracji i uprawnień. Zaakceptowany plan opisuje zamiar, bez technicznego ograniczania działań.

# Kontekst projektu

Wybrane informacje o produkcie, strukturze kodu i zasadach pracy, potrzebne do bieżącego zadania.

## Cel doboru

Mapa koszyka gościa i logowania wymaga źródeł o tej ścieżce, jej danych i sposobie kontroli.

## Reguły wspólne

Konwencje zespołu obowiązują także przy innym zadaniu. Bieżący stan implementacji wymaga aktualnego odczytu.

# Źródła kontekstu

| Źródło                                | Rola przy rozpoznaniu koszyka                         |
|---------------------------------------|-------------------------------------------------------|
| <code>README.md</code>                | Uruchomienie produktu, reset stanu i komendy kontroli |
| <code>package.json</code>             | Dostępne skrypty i deklarowane zależności             |
| <code>docs/product-contract.md</code> | Udokumentowane zachowanie produktu                    |
| Aktualny kod                          | Odpowiedzialność za odczyt, zmianę i zapis koszyka    |
| Testy i obserwacja                    | Zachowanie w konkretnym stanie i dla danego wejścia   |

Polityka danych: kontekst zawiera fikcyjne dane warsztatu. Sekrety i dane klientów pozostają poza nim.

# Instrukcje projektu

Trwałe reguły kierujące pracą agenta w repozytorium, stosowane zgodnie z formatem i zakresem narzędzia.

## Treść reguły

Nieoczywista konwencja zespołu, właściwe źródło albo wymagany sposób weryfikacji.

## Przykład

Opis koszyka zawiera odwołania do kodu. Wyniki wykonanych kontroli są oddzielone od kontroli zaplanowanych.

# Formaty instrukcji

| Format                                       | Typowe zastosowanie                                    |
|----------------------------------------------|--------------------------------------------------------|
| <code>.github/copilot-instructions.md</code> | Wspólne reguły projektu dla Copilota                   |
| <code>*.instructions.md</code>               | Reguły Copilota dla dopasowanych plików lub zadań      |
| <code>AGENTS.md</code>                       | Wspólny format obsługiwany m.in. przez Codex i Copilot |
| <code>CLAUDE.md</code>                       | Instrukcje projektu dla harnessu Claude                |

Format, lokalizacja i sposób wczytania zależą od wybranego harnessu oraz ustawień sesji.

# Instrukcje warunkowe

Reguły dołączane do wybranych plików lub rodzaju zadania zamiast do każdej pracy w projekcie.

W formacie Copilota `applyTo` dopasowuje ścieżki względem katalogu workspace.

Przykład `.instructions.md`

```
---
```

```
applyTo: "**/*.tsx"
```

```
---
```

Przy opisie komponentu wskaż źródło danych  
i zdarzenie zmieniające jego stan.

# Reguła i wynik

|                    |                                                                             |
|--------------------|-----------------------------------------------------------------------------|
| Informacja         | Miejsce i czas życia                                                        |
| Stała reguła pracy | Instrukcja projektu: mapa koszyka wskazuje źródła ustaleń                   |
| Wiedza o produkcie | Dokumentacja domeny: zachowanie koszyka po logowaniu, potwierdzone w kodzie |
| Wynik rozpoznania  | Mapa bieżącej wersji: miejsca odpowiedzialności i otwarte pytania           |
| Wynik kontroli     | Notatka próby: stan wejściowy, wykonana operacja i obserwacja               |

Próba w nowej rozmowie obejmuje obecność instrukcji w źródłach oraz zastosowanie reguły w wyniku.

# Instrukcja i prompt

Prompt file to zapisane polecenie wielokrotnego użytku, uruchamiane dla wybranego zadania.

| Artefakt    | Rola                                     | Przykład                                                            |
|-------------|------------------------------------------|---------------------------------------------------------------------|
| Instrukcja  | Wspólna reguła pracy                     | Mapa wskazuje źródło każdego istotnego ustalenia                    |
| Polecenie   | Cel bieżącej pracy                       | Odtworzenie ścieżki koszyka gościa przez logowanie                  |
| Prompt file | Powtarzalne polecenie z podanym zakresem | Rozpoznanie wskazanej funkcjonalności i kontrola jednego zachowania |

W VS Code pliki `.prompt.md` obsługuje Local agent. Agent Host wymaga innego sposobu reuse.

# Codebase

Kod źródłowy projektu wraz z konfiguracją, testami i zależnościami potrzebnymi do działania produktu.

## Granica rozpoznania

Jedna funkcjonalność: koszyk gościa i przejście przez logowanie.

## Mapa odpowiedzialności

Pliki i symbole kodu powiązane z wejściem użytkownika, stanem danych oraz widocznym wynikiem.

# Wyszukiwanie odpowiedzialności

Wyszukiwanie semantyczne znajduje kod po znaczeniu pytania, także bez znajomości dokładnej nazwy.

|                       |                                                                |
|-----------------------|----------------------------------------------------------------|
| Sposób szukania       | Zastosowanie przy rozpoznaniu koszyka                          |
| Znaczenie pytania     | „Gdzie koszyk reaguje na zalogowanie użytkownika?”             |
| Dokładny tekst        | Odczyt znalezionej nazwy lub komunikatu i wyszukanie jego użyc |
| Definicje i wywołania | Ustalenie, kto wywołuje operację i kto korzysta z wyniku       |
| Odczyt kodu           | Potwierdzenie roli miejsca znalezioneego przez wyszukiwanie    |

# Przeptyw wykonania

Przeptyw danych opisuje przekazywanie i przekształcanie wartości. Przeptyw sterowania opisuje kolejność operacji i wybór gałęzi.

Dane koszyka

Źródło pozycji, odczyt ilości, zmiana stanu i miejsce zapisu.

Decyzje programu

Rozpoznanie gościa lub konta, warunek uruchomienia operacji oraz zachowanie po błędzie odczytu.

# Reverse engineering

Odtwarzanie istniejącego zachowania i reguł produktu na podstawie jego implementacji oraz obserwacji działania.

## Odczyt implementacji

Kod wskazuje możliwe przejścia między stanami koszyka i warunki ich wykonania.

## Obserwacja działania

Odtwarzalny przykład wiąże wybrane przejście z konkretnym wejściem i widocznym wynikiem.

# Mapa z dowodami

| Element mapy     | Co pozwala sprawdzić ustalenie                          |
|------------------|---------------------------------------------------------|
| Odpowiedzialność | Rzeczywisty plik i symbol, ich rola w obsłudze koszyka  |
| Połączenie       | Wywołanie albo przekazana wartość wiążąca dwa miejsca   |
| Reguła przejścia | Warunek w kodzie, np. rozpoznanie zalogowanego konta    |
| Zachowanie       | Stan wejściowy, wykonana operacja i zaobserwowany wynik |
| Luka             | Niepotwierdzony element i potrzebne źródło lub próba    |

# Git i dokumentacja

| Źródło                | Pytanie, na które odpowiada                         |
|-----------------------|-----------------------------------------------------|
| Aktualny kod          | Jak teraz przebiega odczyt koszyka po logowaniu?    |
| Dokumentacja produktu | Jak opisano oczekiwane zachowanie i użyte pojęcia?  |
| Historia Git          | Kiedy i w jakim zakresie zmieniono ten fragment?    |
| Opis commita          | Jak autor uzasadnił zmianę w tamtym momencie?       |
| Kontrola zachowania   | Co zachodzi dla podanego wejścia w bieżącej wersji? |

Rozbieżność opisu i kodu pozostaje jawnym punktem do wyjaśnienia.

# Polecenie eksploracji

Wyjaśnij istniejące zachowanie <funkcjonalność>.  
Granica analizy: od <wejście> do <widoczny wynik>.

Powiąz odpowiedzialności, dane i warunki przejść.  
Przy ustaleniach podaj rzeczywiste pliki i symbole.  
Oddziel fakty z kodu, założenia i otwarte pytania.  
Zaproponuj małą próbę istotnego zachowania.  
Raportuj wykonane operacje i ich wyniki osobno od planu.

Zmiany implementacji pozostają poza zakresem analizy.

# Wymaganie i plan

Kontrakt zachowania przed zmianą kodu

# Kontrakt zachowania

Sprawdzalny opis warunków wejściowych, oczekiwanego wyniku i wyjątków zmiany.

## Polityka dostawy

„Darmowy kurier dla większych zakupów” określa potrzebę. Kontrakt ustala, kiedy reguła działa i jaką kwotę otrzymuje klient.

## Podstawa kontroli

Przykład wejścia i oczekiwanego wyniku pozwala ocenić propozycję AI oraz późniejszą implementację.

# Braki i założenia

| Status informacji  | Przykład przy dostawie                         | Podstawa decyzji                      |
|--------------------|------------------------------------------------|---------------------------------------|
| Wymaganie          | Darmowy kurier dla większych zakupów           | Aktualne zgłoszenie                   |
| Brak reguły        | Która wartość koszyka jest podstawą warunku?   | Odpowiedź właściciela wymagania       |
| Założenie          | Hipoteza: reguła obejmuje każdą metodę dostawy | Jawne potwierdzenie albo odrzucenie   |
| Sprzeczność źródeł | Jeżeli opis reguły różni się od zgłoszenia     | Rozstrzygnięcie zakresu i aktualności |

Nierozstrzygnięta reguła wstrzymuje zależną implementację.

# Mapa wpływu

Powiązanie zmienianej reguły z odpowiedzialnym kodem i jego konsumentami.

Każde miejsce ma odwołanie do rzeczywistego pliku lub symbolu oraz powód uwzględnienia.

## Ślad kwoty

1. **Źródło** — gdzie powstaje kwota dostawy?
2. **Konsumenci** — kto ją odczytuje, pokazuje lub zapisuje?
3. **Warunki** — co wybiera właściwą gałąź reguły?
4. **Kontrole** — które istniejące testy dotyczą tych miejsc?

# Zależności i regresja

Zależność przenosi skutek zmiany do innego miejsca. Regresja to pogorszenie zachowania, które ma pozostać zachowane.

| Powiązanie         | Możliwy skutek zmiany dostawy                | Kontrola w planie                |
|--------------------|----------------------------------------------|----------------------------------|
| Wspólna kalkulacja | Inny konsument otrzymuje nową kwotę          | Lista odczytów i zapisów         |
| Prezentacja kwoty  | Klient widzi kwotę różną od rozliczanej      | Porównanie wartości w przepływie |
| Warunek reguły     | Zmieniona gałąź obejmuje dodatkowy przypadek | Przypadek poza nowym zakresem    |

# Małe kroki

## Cel i punkt kontroli

Krok ma ograniczony zakres i dowód pozwalający ocenić jego wynik przed następną zależną zmianą.

1. **Ustalenie reguły** — potwierdzony kontrakt i otwarte pytania.
2. **Kontrola bazowa** — znane zachowanie kodu przed zmianą.

3. **Wydzielona zmiana** — jeden cel, ograniczone miejsca, kontrola odpowiadająca celowi.
4. **Spójność przepływu** — sprawdzenie zależnych miejsc i zachowania poza nowym zakresem.

# Plan testowania

Plan testowania wiąże przypadek z oczekiwanym wynikiem i sposobem sprawdzenia.

| Przypadek | Przykład dla dostawy                               | Co rozstrzyga wynik                        |
|-----------|----------------------------------------------------|--------------------------------------------|
| Typowy    | Zwykły koszyk spełniający warunki promocji         | Kwota dostawy wynikająca z kontraktu       |
| Brzegowy  | Wartość poniżej, na i powyżej granicy reguły       | Dokładne znaczenie warunku granicznego     |
| Regresji  | Dotychczasowe zachowanie poza zakresem promocji    | Jawne kryterium zachowania tej reguły      |
| Spójności | Ta sama transakcja w kolejnych miejscach przepływu | Zgodność prezentowanej i rozliczanej kwoty |

# Review planu

Review planu ocenia zakres i sposób sprawdzenia zmiany przed kodowaniem.

Zadanie: analiza <zgłoszenie>, bez zmian kodu.

Źródła: <wymaganie>, <reguły>, <repo>.

1. Oddziel fakty, braki i założenia.
2. Odtwórz zależności z odwołaniami do kodu.
3. Zaproponuj kontrakt i przypadki sprawdzające.
4. Dla kroków podaj zależności i punkty kontroli.
5. Wskaż luki blokujące przyjęcie planu.
6. Oceń zgodność z wymaganiem, zakres i testy.

Wskaż potrzebne poprawki.

Wynik: plan do decyzji człowieka.

# Implementacja z AI

Przyjęty plan i kontrolowane zmiany

# Kontrolowana iteracja

Mały etap implementacji z własnym celem, ograniczonym zakresem i kontrolą zachowania przed następnym etapem.

## Zakres etapu

Jedna część reguły dostawy może wymagać kilku plików.  
Liczbę edycji wyznacza odpowiedzialność kodu.

## Warunek kontynuacji

Wynik kontroli odpowiada na cel etapu. Rozbieżność wymaga wyjaśnienia przed następną zmianą.

# Wzorce i istniejące API

| Decyzja                       | Kontekst dla AI                           | Dowód w zmianie                   |
|-------------------------------|-------------------------------------------|-----------------------------------|
| Sposób obliczania dostawy     | Analogiczna reguła kwotowa w repo         | Uzasadnione użycie wzorca         |
| Obsługa danych checkout       | Istniejący kontrakt API i jego konsumenci | Zgodność formatu wejścia i wyniku |
| Zastosowanie biblioteki       | Zainstalowana wersja i użycie w projekcie | Wywołanie dostępnego API          |
| Nowa zależność lub abstrakcja | Luka, której obecny kod nie pokrywa       | Konkretne uzasadnienie wyjątku    |

# Jedna reguła, kilku konsumentów

| Granica                      | Pytanie kontrolne                        | Przykład kontroli dostawy                     |
|------------------------------|------------------------------------------|-----------------------------------------------|
| Reguła i jej wejścia         | Czy obliczenie realizuje kontrakt?       | Wartości poniżej, na i powyżej progu          |
| Serwer i prezentacja         | Czy ten sam przypadek daje zgodne kwoty? | Porównanie wyniku serwera z ekranami checkout |
| Obliczenie i zapis           | Czy zamówienie zachowuje ustaloną kwotę? | Porównanie checkout z zapisanym zamówieniem   |
| Zmieniana i pozostałe reguły | Czy dotychczasowy wariant nadal działa?  | Kontrola innego sposobu dostawy               |

# Ocena diff

Diff pokazuje różnicę między wersjami plików. Umożliwia ocenę zakresu i celu każdej istotnej edycji.

| Fragment zmiany           | Pytanie przed przyjęciem                    |
|---------------------------|---------------------------------------------|
| Reguła kosztu dostawy     | Który zapis kontraktu realizuje ta edycja?  |
| Konsument kwoty           | Dlaczego ten plik wymaga zmiany?            |
| Test lub kontrola         | Jakie zachowanie potwierdza wykonany wynik? |
| Plik poza planem          | Jaki odkryty wpływ uzasadnia rozszerzenie?  |
| Przy okazji zmieniony kod | Czy usunięcie tej edycji naruszy wymaganie? |

# Konkretny feedback

Feedback wiąże obserwowany wynik z kontraktem i określa zakres korekty oraz ponownej kontroli.

Przypadek: <wartość koszyka i sposób dostawy>.

Obserwacja: <wynik i miejsce wykonanej kontroli>.

Oczekiwanie: <zachowanie z kontraktu>.

Ustal, z czego wynika rozbieżność.

Popraw tylko elementy potrzebne do tego wymagania.

Powtórz kontrolę przypadku oraz powiązaną regresję.

Pokaż wynik kontroli i różnicę po korekcie.

# Procedura implementacji

Procedura utrwała sposób wykonania przyjętego planu. Szablon promptu oddziela stałe kroki od parametrów konkretnej zmiany.

```
Wejście: <kontrakt>, <plan>, <reguły repo>.  
Wskaż wzorce i istniejące API potrzebne do zmiany.  
Wybierz mały etap i zapowiedz jego kontrolę.  
Wykonaj etap. Uruchom zapowiedzianą kontrolę.  
Pokaż diff oraz wynik względem kontraktu.  
Przy rozbieżności ustal przyczynę, popraw i powtórz kontrolę.  
Przy nowym zakresie przedstaw uzasadnienie do decyzji.  
Na koniec podaj stan wykonania i niewykonane kontrole.
```

# Test, asercja, mock

| Pojęcie | Definicja                                                         | Zastosowanie                                           |
|---------|-------------------------------------------------------------------|--------------------------------------------------------|
| Test    | Powtarzalna kontrola zachowania dla ustalonego wejścia i warunków | Sprawdzenie operacji koszyka względem wymagania        |
| Asercja | Warunek określający, czy wynik spełnia oczekiwanie                | Porównanie skutku operacji z oczekiwanym zachowaniem   |
| Mock    | Kontrolowany zastępnik zależności używany w teście                | Ustalenie odpowiedzi zależności bez zewnętrznej usługi |

# TDD

Rozwijanie zachowania w krótkim cyklu, w którym test wymagania poprzedza implementację.

## Red

Test nie przechodzi, ponieważ wymagane zachowanie jeszcze nie działa.

## Green

Najmniejsza potrzebna implementacja sprawia, że test przechodzi.

## Refactor

Uproszczenie kodu zachowuje zachowanie potwierdzone przez testy.

# Przypadki wynikają z wymagania

Każdy przypadek łączy wejście, warunki i oczekiwany wynik z kontraktu.

| Rodzaj przypadku | Rola                                                 | Przykład dla koszyka                                               |
|------------------|------------------------------------------------------|--------------------------------------------------------------------|
| Typowy           | Potwierdza zwykłe użycie zgodne z kontraktem         | Zmiana ilości przy poprawnych danych                               |
| Brzegowy         | Sprawdza granicę, przy której może zmienić się wynik | Ilość poniżej, na i powyżej granicy opisanej w wymaganiu           |
| Regresyjny       | Chroni dotychczasowe zachowanie po zmianie           | Poprawna operacja koszyka, której nowa reguła nie powinna zmieniać |

# Poziom testu określa granicę dowodu

| Poziom      | Co sprawdza                                                                    | Ograniczenie                                       |
|-------------|--------------------------------------------------------------------------------|----------------------------------------------------|
| Unit        | Jednostkę logiki w izolacji, np. ocenę ilości w koszyku                        | Nie potwierdza podłączenia reguły do operacji      |
| Integration | Współpracę elementów, np. wejście HTTP, logikę koszyka i rzeczywisty zapis     | Nie potwierdza zachowania kontrolek w przeglądarce |
| E2E         | Wybrany przepływ użytkownika przez aplikację, np. zmianę ilości w przeglądarce | Obejmuje tylko wykonany scenariusz i stan          |

# Dowód każdego etapu TDD

| Etap     | Dowód związany z wymaganiem                                                                                              |
|----------|--------------------------------------------------------------------------------------------------------------------------|
| Red      | Konkretna asercja wykrywa brak zachowania. Test dociera do badanej reguły. Błąd konfiguracji nie stanowi takiego dowodu. |
| Green    | Te same dane i oczekiwanie przechodzą po zmianie realizującej wymaganie.                                                 |
| Refactor | Te same kontrole przechodzą po uproszczeniu kodu. Oczekiwane zachowanie pozostaje zgodne z kontraktem.                   |

# Niezależny dowód wymagania

Niezależny dowód ma podstawę w kontrakcie, a oczekiwanego wyniku nie wyznacza badana implementacja.

| Element testu    | Podstawa oceny                                        | Luka w dowodzie                                         |
|------------------|-------------------------------------------------------|---------------------------------------------------------|
| Oczekiwany wynik | Przykład rozstrzygnięty z wymagania                   | Obliczenie oczekiwania tą samą funkcją, którą bada test |
| Asercja          | Skutek operacji koszyka wymagany przez kontrakt       | Sprawdzenie samego braku błędu                          |
| Mock             | Zastąpienie zależności przy zachowaniu badanej reguły | Gotowa decyzja mocka omija regułę podlegającą kontroli  |

# Siła testu

Kontrolowana błędna zmiana to odwracalna modyfikacja, która celowo narusza wymaganie. Ujawnia, czy test wykrywa to naruszenie.

## Próba

Po Green niewielka zmiana narusza jeden ustalony warunek koszyka. Te same testy oceniają jej skutek.

## Ocena

Niepowodzenie właściwej asercji potwierdza wykrycie tej zmiany. Zielony wynik wskazuje lukę w kontroli. Po cofnięciu próby ponowne przejście testów potwierdza odtworzenie stanu.

# Szablon walidacji

Szablon promptu utrwała procedurę oceny testów. Parametry pozwalają użyć jej przy kolejnym wymaganiu.

```
Wejście: <kontrakt>, <testy>, <zakres zmiany>.  
Wyprowadź przypadek typowy, brzegowy i regresyjny.  
Dla każdego podaj wejście, oczekiwanie i jego źródło.  
Uzasadnij poziom testu i wskaż granicę dowodu.  
Oceń niezależność oczekiwań, asercje i zakres mocków.  
Uruchom kontrole. Rozróżnij plan od wykonanego wyniku.  
Wskaż dowody Red, Green i kontroli po Refactor.  
Oceń test odwracalną błędną zmianą. Cofnij próbę.  
Podaj wykryte luki i niewykonane kontrole.
```

# Reprodukcja i diagnoza

| Pojęcie     | Definicja                                                   | Zastosowanie                                            |
|-------------|-------------------------------------------------------------|---------------------------------------------------------|
| Objaw       | Obserwowana niezgodność z oczekiwanym zachowaniem           | Kwota płatności różna od kwoty zamówienia               |
| Reprodukcja | Powtórzenie objawu przy zapisanym wejściu, krokach i stanie | Umożliwia porównanie płatności przed poprawką i po niej |
| Przyczyna   | Mechanizm, który wywołuje objaw                             | Wyznacza uzasadniony zakres poprawki checkout           |
| Diagnoza    | Ustalenie przyczyny na podstawie obserwacji i kontroli      | Wiąże rozbieżność kwot z działaniem kodu                |

# Objaw i przyczyna

Materiał z reprodukcji

Pytanie kontrolne

Słuchawki, kurier, kroki zakupu, stan mocka i wersja kodu

Czy kolejne uruchomienie zachowuje te same warunki?

Kwoty z ekranu płatności i zamówienia

W którym momencie tej samej operacji widać rozbieżność?

Wartości i decyzje na granicach warstw checkout

Gdzie po raz pierwszy zachowanie odbiega od oczekiwania?

Podejrzane miejsce w kodzie

Która kontrola wiąże jego działanie z powtarzalnym objawem?

Znikający komunikat

Czy porównanie kwot nadal pokazuje rozbieżność?

# Log

Zapis zdarzenia lub wartości podczas wykonania programu.  
Pomaga powiązać obserwację z konkretną operacją.

## Kontekst płatności

Czas, wejście zakupu, stan mocka i wersja kodu wiążą wpis z reprodukcją.

# Stack trace

Ślad wywołań prowadzących do miejsca utworzenia błędu.  
Pomaga odnaleźć kod wymagający sprawdzenia.

## Zakres dowodu

Lokalizacja w śladzie wymaga porównania z wejściem i kodem. Rozbieżność kwot może wystąpić bez wyjątku.

# Hipoteza i próba

Hipoteza to możliwe wyjaśnienie objawu. Próba rozstrzygająca daje wynik, który wspiera lub podważa wyjaśnienie.

Priorytet ma mała kontrola rozróżniająca konkurujące hipotezy.

| Hipoteza robocza       | Pytanie kontrolne                                            | Próba                                          |
|------------------------|--------------------------------------------------------------|------------------------------------------------|
| Błąd prezentacji kwoty | Czy ekran pokazuje otrzymaną wartość dla tej samej operacji? | Porównanie danych ekranu z widoczną kwotą      |
| Błąd naliczania kwoty  | Czy wartość zmienia się zgodnie z regułą zakupu?             | Śledzenie wejścia i wyniku obliczenia w kodzie |

# Kolejne próby mają granice

Agent prowadzi serię prób w uzgodnionej ścieżce checkout. Każda następna próba wynika z poprzedniego wyniku.

| Wynik próby narzędziowej              | Decyzja                                          | Ślad płatności                           |
|---------------------------------------|--------------------------------------------------|------------------------------------------|
| Odczyt rozróżnia hipotezy             | Aktualizacja hipotezy i wybór następnej kontroli | Wejście, narzędzie, obserwowana kwota    |
| Reprodukcja potwierdza mechanizm      | Stop diagnozy, ustalenie zakresu poprawki        | Miejsce w kodzie i wynik próby           |
| Próba nie wnosi nowego dowodu         | Stop serii, zmiana planu                         | Luka w obserwacji checkout               |
| Brakuje potrzebnego stanu lub dostępu | Stop, decyzja człowieka                          | Niewykonana kontrola i brakujący warunek |

# Minimalna poprawka i regresja

Minimalna poprawka usuwa potwierdzony mechanizm w największym uzasadnionym zakresie.

| Kontrola po zmianie           | Porównanie                                       | Dowód                                       |
|-------------------------------|--------------------------------------------------|---------------------------------------------|
| Ta sama reprodukcja płatności | To samo wejście i stan, wersja przed i po        | Wynik wobec uzgodnionego oczekiwania        |
| Potwierdzony mechanizm        | Wartości w miejscu, które wyjaśniało rozbieżność | Ślad pokazujący skutek poprawki             |
| Regresja powiązanego wariantu | Inny sposób dostawy z zachowaniem kontraktu      | Powtarzalny test dotychczasowego zachowania |
| Zakres diff                   | Każda edycja wobec przyczyny lub kontroli        | Uzasadnienie zmienionych miejsc             |

# Prompt diagnozy

Zgłoszenie: <obserwowany objaw>.

Reprodukcja: <wejście, kroki, stan, wersja>.

Oczekiwanie: <reguła i jej źródło>.

Materiały: <kod, log, stack trace, jeśli występuje>.

Oddziel obserwacje od hipotez.

Ułóż hipotezy i wybierz małą próbę rozstrzygającą.

Po wyniku zaktualizuj hipotezę. Pokaż dowód mechanizmu.

Zatrzymaj serię przy braku nowego dowodu lub warunków próby.

Po potwierdzeniu zaproponuj minimalną poprawkę.

Powtórz reprodukcję i kontrolę regresji.

Podaj wyniki przed/po oraz niewykonane kontrole.

# Code review

Ocena konkretnej zmiany kodu względem wymagania, z uwzględnieniem jej wpływu na produkt i istniejące zachowanie.

## Cel review

Uwagi wspierają decyzję, czy zmiana dostawy jest gotowa do przyjęcia.

## Rola człowieka

Człowiek przyjmuje, odrzuca lub kieruje uwagę do weryfikacji. Poprawka kodu wymaga osobnej decyzji.

# Wymaganie wyznacza ocenę diff

| Wejście    | Pytanie kontrolne      | Przykład dostawy                                   |
|------------|------------------------|----------------------------------------------------|
| Wymaganie  | Co produkt ma robić?   | Wersja kontraktu i oczekiwane zachowanie           |
| Diff       | Co zmienił autor?      | Reguła dostawy i miejsca jej użycia                |
| Konsumenci | Co może odczuć zmianę? | Kwota w checkout i zapis zamówienia                |
| Kontrole   | Jakie dowody istnieją? | Treść testów, wejścia koszyka i rzeczywiste wyniki |

# Wpływ wyznacza priorytet uwagi

| Obszar                   | Przedmiot oceny                       | Pytanie o dostawę                              |
|--------------------------|---------------------------------------|------------------------------------------------|
| Poprawność (correctness) | Zgodność wyniku z wymaganiem          | Czy kwota odpowiada regule dla tego koszyka?   |
| Regresja                 | Zachowanie poza zakresem nowej reguły | Czy dotychczasowy sposób dostawy nadal działa? |
| Czytelność               | Łatwość odtworzenia intencji kodu     | Czy warunek pokazuje regułę bez zgadywania?    |

Priorytet wynika z wpływu na wynik i użytkownika. Sama różnica stylu uzasadnia niższy priorytet.

# Test w review rozstrzyga wymaganie

Ślad w teście

Pytanie jakościowe

Oczekiwanie

Czy oczekiwana kwota wynika z kontraktu dostawy?

Asercja

Czy test porównuje właściwą kwotę, zamiast tylko sprawdzać powodzenie?

Przypadki koszyka

Jaka błędna realizacja wymagania mogłaby nadal przejść te testy?

Dane testowe

Czy przygotowane dane ukrywają zachowanie istotne dla dostawy?

# Granica zaufania

Miejsce przyjęcia danych, których poprawności dana część aplikacji nie może zakładać.

## Wejście dostawy

Review obejmuje miejsce walidacji danych koszyka przed wyliczeniem dostawy.

## Obsługa błędu

Błędne dane checkout wymagają jawnej reakcji. Komunikat dla klienta nie ujawnia szczegółów implementacji.

# Dowód określa status uwagi

Potwierdzona uwaga ma sprawdzony skutek. Ryzyko opisuje możliwy skutek bez wystarczającego dowodu.

| Status              | Ślad dla zmiany dostawy                                    | Decyzja człowieka         |
|---------------------|------------------------------------------------------------|---------------------------|
| Potwierdzona uwaga  | Miejsce w diff, wejście koszyka, skutek, przykład i dowód  | Przyjęcie z uzasadnieniem |
| Ryzyko              | Podejrzenie wolniejszego checkout bez pomiaru              | Dodatkowa kontrola        |
| Niepoparta uwaga    | Zarzut bez związku z diff lub regułą dostawy               | Odrzucenie z przyczyną    |
| Brak istotnych uwag | Zakres review dostawy, sprawdzone przypadki, wyniki i luki | Ocena gotowości           |

# Procedura review

Stały przebieg review z parametrami konkretnej zmiany.

Wejście: <wymaganie>, <diff>, <konsumenci>, <testy>.

Oceń zgodność i regresje względem wymagania.

Nadaj priorytet skutkom, potem czytelności.

Oceń, czy asercje niezależnie sprawdzają wymaganie.

Sprawdź granice zaufania i obsługę błędów.

Dla uwagi podaj miejsce, skutek, przykład i dowód.

Oddziel potwierdzone uwagi od ryzyk.

Przy braku istotnych uwag pokaż zakres i kontrole.

Oznacz wykonane kontrole i luki. Nie poprawiaj kodu.

Przedstaw podstawę decyzji człowieka o gotowości.

# Refactoring

Zmiana wewnętrznej struktury kodu przy zachowaniu uzgodnionego zachowania.

## Problem utrzymania

Osobne wyliczenia dostawy w widoku i zamówieniu zwiększają ryzyko zmiany reguły tylko w jednym miejscu.

## Cel uproszczenia

Wyraźne miejsce decyzji o koszcie ułatwia spójną zmianę wyliczeń obu konsumentów.

# Charakterystyka zachowania

Charakterystyka opisuje, co istniejący kod robi dla ustalonych wejść i warunków. Stanowi punkt odniesienia przed przebudową.

| Element            | Ślad dla kosztu dostawy                               |
|--------------------|-------------------------------------------------------|
| Wejścia            | Całkowita, nieujemna suma groszy i obsługiwana metoda |
| Wyniki             | Zarejestrowana kwota dla każdego ustalonego wejścia   |
| Konsumenci         | Wyliczenie dla widoku i dla zamówienia                |
| Chroniony kontrakt | Uzgodniony sposób wywołania oraz wynik obu wyliczeń   |

# Równoważność i nowa funkcja

Równoważność oznacza te same obserwowalne wyniki dla uzgodnionych wejść i warunków.

| Rodzaj zmiany | Kryterium przyjęcia                          | Przykład dostawy                                   |
|---------------|----------------------------------------------|----------------------------------------------------|
| Refactoring   | Dotychczasowe wyniki i publiczne wywołania   | Uproszczenie wyliczenia bez nowej polityki dostawy |
| Nowa funkcja  | Nowy kontrakt i odpowiadające mu oczekiwania | Zmiana kosztu dostawy dla określonego przypadku    |

Zmiana oczekiwań wymaga osobnego uzgodnienia wymagania.

# Czytelność służy kolejnej zmianie

| Kryterium        | Pytanie kontrolne dla reguły dostawy                                |
|------------------|---------------------------------------------------------------------|
| Odpowiedzialność | Czy reguła ma wyraźne miejsce, oddzielone od wyświetlenia i zapisu? |
| Czytelność       | Czy nazwy pokazują sens danych i decyzji o koszcie?                 |
| Zakres diff      | Czy każda edycja służy uproszczeniu wskazanej duplikacji?           |
| Wzorzec projektu | Czy wydzielenie korzysta z istniejącego sposobu organizacji kodu?   |

# Dowód obejmuje oba wyliczenia

| Granica kontroli | Oczekiwane zachowanie                                 | Dowód przed i po                                              |
|------------------|-------------------------------------------------------|---------------------------------------------------------------|
| Obliczenie       | Ta sama kwota dla tych samych poprawnych wejść        | Testy charakteryzujące regułę dostawy                         |
| Konsumenci       | Dotychczasowy sposób użycia przez widok i zamówienie  | Kontrola publicznych wywołań obu wyliczeń                     |
| Regresja         | Zachowane dotychczasowe zachowanie w obszarze dostawy | Testy związane z uproszczonym fragmentem i jego zależnościami |

# Procedura refactoringu

Prompt utrwała metodę. Parametry pozwalają zastosować ją do kolejnego problemu utrzymania.

Wejście: <problem>, <kod>, <chroniony kontrakt>, <kontrole>.

Wskaż duplikację i konkretny koszt kolejnej zmiany.

Odtwórz zachowanie dla uzgodnionych wejść konsumentów.

Uruchom kontrole przed edycją i zachowaj wyniki.

Zaproponuj najmniejsze uproszczenie odpowiedzialności.

Zachowaj publiczne wywołania i oczekiwane wyniki.

Po zmianie ponów te same kontrole.

Przejrzyj diff i uzasadnij każdą edycję.

Zwróć porównanie przed/po, wynik kontroli oraz luki.

# Skill

Pakiet instrukcji dla wyspecjalizowanego sposobu pracy, dostępny do wczytania przy pasującym zadaniu. Zawiera plik `SKILL.md` i opcjonalne zasoby.

## Funkcja procedury

Analiza nowej zmiany koszyka korzysta z tych samych kroków ustalania wymagania, źródeł i wpływu na kod.

## Zasoby pomocnicze

Szablon wyniku lub przykład kontroli może być osobnym plikiem wskazanym w procedurze.

# Funkcja wyznacza formę utrwalenia

| Forma       | Czas życia i funkcja                        | Przykład koszyka                                               |
|-------------|---------------------------------------------|----------------------------------------------------------------|
| Prompt      | Bieżące zadanie i jego parametry            | Rozpoznanie zakresu nowego zgłoszenia                          |
| Prompt file | Polecenie uruchamiane ponownie              | Eksploracja wskazanej funkcjonalności                          |
| Instrukcja  | Stała lub warunkowa reguła projektu         | Zmiana wymaga testów i review; dokumentacja zawiera ich wyniki |
| Skill       | Procedura z krokami, kryteriami i wyjątkami | Analiza zmiany, implementacja albo walidacja                   |

# Każda potrzeba ma własny plik

| Kiedy użyć                                          | Plik                                                           | Przykład ShopEasy                     |
|-----------------------------------------------------|----------------------------------------------------------------|---------------------------------------|
| To samo polecenie uruchamiane dla wybranego zadania | <b>Prompt file</b><br><code>.github/prompts/*.prompt.md</code> | Analiza wskazanej ścieżki koszyka     |
| Reguła obowiązująca w pracy nad projektem           | <b>Instrukcja</b> <code>AGENTS.md</code>                       | Każde ustalenie wskazuje źródło       |
| Powtarzalna procedura z krokami i kontrolą wyniku   | <b>Skill</b> <code>.github/skills/*/SKILL.md</code>            | Analiza zmiany od zgłoszenia do planu |
| Stała rola, zakres działań i narzędzia              | <b>Custom agent</b><br><code>.github/agents/*.agent.md</code>  | Analizy bada kod i zwraca luki        |

# Plik procedury

`name` odpowiada nazwie folderu. `description` opisuje funkcję i sytuację zastosowania. Treść zawiera właściwą procedurę.

Przykład w VS Code: `.github/skills/analize-change/SKILL.md` .

Obsługiwane lokalizacje i aktywacja zależą od wybranego harnessu.

```
---  
name: analize-change  
description: Analiza małej zmiany  
           przed implementacją.  
---  
  
## Wejście  
<zgłoszenie>, <źródła>, <zakres>  
## Przebieg  
<kroki>, <kontrola>, <wyjątki>  
## Wynik  
<kontrakt>, <plan>, <luki>
```

# Workflow łączy wyniki etapów

Workflow to uporządkowany przebieg pracy, w którym wynik etapu staje się wejściem następnego. Procedury Skills mogą realizować poszczególne etapy.

| Etap             | Wejście                      | Wynik dla zmiany koszyka              |
|------------------|------------------------------|---------------------------------------|
| Analiza          | Zgłoszenie i źródła produktu | Wymaganie, wpływ na kod i jawne luki  |
| Planowanie       | Rozstrzygnięte wymaganie     | Małe etapy oraz sposób kontroli       |
| Implementacja    | Przyjęty plan i reguły repo  | Diff i wyniki kontroli etapów         |
| Walidacja/review | Wymaganie, diff i testy      | Ocena spełnienia kontraktu z dowodami |

# Parametry zastępują szczegóły jednej historii

| Element   | Stały sposób pracy                 | Parametr nowej próby                 |
|-----------|------------------------------------|--------------------------------------|
| Wymaganie | Powiązanie decyzji ze źródłem      | Zgłoszenie opróżnienia koszyka       |
| Zakres    | Ustalenie granicy wpływu           | Obszar produktu i dane objęte zmianą |
| Kontrola  | Porównanie zachowania z kontraktem | Anulowanie i potwierdzenie operacji  |
| Rezultat  | Oddzielenie faktów, luk i dowodów  | Pliki oraz wyniki bieżącej próby     |

Kwoty i progi dostawy należą do kontraktu tamtej zmiany, poza ogólną procedurą.

# Kryteria sterują przejściem i stopem

Kryterium przejścia określa wynik potrzebny do następnego etapu. Warunek stopu wskazuje, kiedy procedura kończy bieżący przebieg.

Wejście: <wymaganie>, <źródła>, <zakres>.

Gotowość kroku: <potrzebne decyzje i materiały>.

Przejście: <obserwowalny wynik spełniający kryterium>.

Przy braku reguły wskaż lukę i potrzebną decyzję.

Zatrzymaj etap zależny od brakującej reguły.

Stop: <osiągnięty cel> albo <nierozstrzygnięta zależność>.

Wyjście: <dowody>, <luki>, <następny możliwy krok>.

# Automatyzacja

Powtarzalny proces uruchamiany na parametrach, z określonym wynikiem, kontrolą i obsługą wyjątków.

## Warunek gotowości

Procedura daje ocenialny rezultat na różnych zadaniach. Jej wejścia, kontrola i warunki stopu są ustalone.

## Przykład procesu

Dla podanego kontraktu koszyka i diff wykonywane są wskazane kontrole. Raport zachowuje wyniki oraz nierozstrzygnięte luki.

# Generalizacja wymaga innego zadania

Generalizacja procedury to przeniesienie sposobu pracy na zadanie o innym wymaganiu.

| Własność procedury       | Dowód w próbie koszyka                                   |
|--------------------------|----------------------------------------------------------|
| Niezależność od historii | Wymaganie koszyka jest ustalane z jego aktualnych źródeł |
| Dostosowanie kontroli    | Sprawdzone zachowania wynikają z nowego kontraktu        |
| Obsługa braków           | Próba z brakującą regułą ujawnia zależny etap            |
| Współpraca etapów        | Implementacja lub walidacja korzysta z rezultatu analizy |
| Granica zastosowania     | Wynik wskazuje, co zadziało i co wymaga zmiany procedury |

# Odkrycie, wczytanie, zastosowanie

Czysta rozmowa korzysta z artefaktu i zgłoszenia, bez instruktażu odziedziczonego z poprzedniego czatu.

|              |                                                                                        |
|--------------|----------------------------------------------------------------------------------------|
| Poziom       | Obserwowalny ślad                                                                      |
| Odkrycie     | Skill widoczny w konfiguracji wybranego harnessu                                       |
| Wczytanie    | Odczyt treści <code>SKILL.md</code> ; potrzebne zasoby odczytane zgodnie z odwołaniami |
| Zastosowanie | Wynik analizy koszyka spełnia kryteria kroków procedury                                |
| Wykonanie    | Aktywność narzędzi i rezultaty potwierdzają użyte kontrole                             |

Sama obecność Skill na liście nie dowodzi wykonania procedury.

# Custom agent

Profil zapisujący rolę, instrukcje i wybór narzędzi dla określonego rodzaju pracy.

## Powtarzalna rola

Profil analizy utrzymuje ten sam zakres przy rozpoznawaniu UI i źródeł filtrów.

## Konkretny wynik

Workshop Analizer zwraca ustalenia z odwołaniami i luki, bez edycji kodu.

# Subagent

Osobno uruchomiony wykonawca podzadania, z własnym kontekstem. Wynik wraca do agenta nadrzędnego.

Agent nadrzędny, czyli koordynator, dzieli pracę i łączy otrzymane wyniki.

## Handoff

Przycisk przekazania przełącza rolę z kontekstem rozmowy i przygotowanym poleceniem.

## Notatka przekazania

Zapis stanu i wyników służy wznowieniu pracy. Sam dokument nie uruchamia subagenta.

# Rola określa wejście i rezultat

| Rola          | Wejście                           | Wynik                        | Narzędzia                      |
|---------------|-----------------------------------|------------------------------|--------------------------------|
| Analiza       | Zgłoszenie, źródła UI/danych      | Ustalenia, odwołania, luki   | read , search                  |
| Implementacja | Kontrakt, zakres zapisu           | Mały diff, wykonane kontrole | read , search , edit , execute |
| Testy         | Kontrakt pustej i niepustej listy | Próby, wyniki, zakres dowodu | read , search , edit , execute |
| Review        | Wymaganie, diff, kontrole         | Uwagi z dowodami, ryzyka     | read , search                  |

# Profil projektu

Plik `.github/agents/analizer.agent.md` udostępnia rolę w workspace.

Opcjonalny nagłówek YAML konfiguruje profil. Treść Markdown opisuje działanie i oczekiwany wynik.

Nazwy narzędzi zależą od wybranego harnessu.

```
---  
name: Workshop Analyzer  
description: Analiza zmiany bez edycji  
tools: ['read', 'search']  
---
```

Odczytaj wskazane źródła UI lub danych.  
Zwróć ustalenia, odwołania i luki.  
Nie zmieniaj plików.

# Zależności wyznaczają kolejność pracy

| Podzadania                                    | Organizacja              | Warunek                                                      |
|-----------------------------------------------|--------------------------|--------------------------------------------------------------|
| Rozpoznanie pustego stanu UI i źródeł filtrów | Równolegle, tylko odczyt | Ustalony stan repo; wynik jednego nie jest wejściem drugiego |
| Implementacja i modyfikacja testów            | Sekwencyjnie             | Każdy zapis korzysta z zakończonego stanu poprzedniego etapu |
| Review gotowej zmiany                         | Po zakończeniu zapisu    | Wymaganie, diff i wyniki kontroli dotyczą tej samej wersji   |

# Kontrakt zlecenia ogranicza niejasność

Rola: analiza źródeł aktywnych filtrów.

Cel: ustal dane potrzebne do opisu pustej listy.

Wejście: <publiczne wymaganie>, <źródła>, <stan repo>.

Zakres: tylko odczyt; bez zmian plików i aplikacji.

Narzędzia: read, search.

Wynik: ustalenia z odwołaniami, luki, potrzebne kontrole.

Stop: brak źródła, sprzeczna reguła lub potrzeba zapisu.

Zwróć konkretną lukę i podstawę decyzji; bez zgadywania.

# Koordynator weryfikuje wyniki

| Otrzymany wynik       | Kontrola nadrzędna                      | Rozstrzygnięcie                        |
|-----------------------|-----------------------------------------|----------------------------------------|
| Ustalenia UI i danych | Odwołania do aktualnych źródeł          | Spójny zakres podsumowania             |
| Sprzeczne propozycje  | Wymaganie i próba na tym samym stanie   | Decyzja z podstawą i jawną luką        |
| Zmiana i review       | Diff, kontrola pustej i niepustej listy | Przyjęcie albo skierowanie do poprawki |

Zgodne odpowiedzi oparte na tych samych źródłach i założeniach nie są niezależną kontrolą. O przyjęciu zmiany decyduje człowiek.

# Delegacja ma zysk i koszt

| Sytuacja                        | Bilans                                                | Decyzja                    |
|---------------------------------|-------------------------------------------------------|----------------------------|
| Oddzielne odczyty UI i danych   | Możliwe krótsze oczekiwanie; osobne pakiety kontekstu | Delegacja wąskich zakresów |
| Ten sam mały fragment           | Powielony odczyt i podobne raporty                    | Jedna rola                 |
| Zależne zmiany filtrów i testów | Uzgadnianie kolejności oraz integracja                | Zapis sekwencyjny          |
| Niejasne wyniki analizy         | Dodatkowe zlecenia i wyjaśnienia                      | Doprecyzowanie kontraktu   |

Krótsza główna rozmowa nie oznacza mniejszego łącznego użycia modeli.

# MCP łączy aplikację ze źródłami

Model Context Protocol (MCP) to standard udostępniania aplikacjom AI narzędzi i danych.

| Element          | Rola                                                 | Przykład                           |
|------------------|------------------------------------------------------|------------------------------------|
| Klient           | Komponent aplikacji komunikujący się z serwerem      | Klient MCP w VS Code               |
| Serwer           | Program udostępniający operacje lub dane             | Lokalny <code>workshop-docs</code> |
| Narzędzie (tool) | Funkcja wywoływana z parametrami                     | <code>search_documents</code>      |
| Zasób (resource) | Dane udostępniane aplikacji jako kontekst do odczytu | Dokument dołączany do rozmowy      |

Dokumenty `workshop-docs` są pobierane przez narzędzia.

# Połączenie, odczyt i aktualność

| Obserwacja      | Co potwierdza                        | Ślad do oceny                                                        |
|-----------------|--------------------------------------|----------------------------------------------------------------------|
| Połączenie      | Komunikację z serwerem               | Stan i log <code>workshop-docs</code>                                |
| Lista narzędzi  | Dostępne operacje                    | <code>search_documents</code> i <code>read_document</code> na liście |
| Wykonany odczyt | Pobranie konkretnej treści           | Wywołanie <code>read_document</code> , parametry i wynik             |
| Aktualne źródło | Obowiązywanie informacji o kontakcie | Wersja, data i zakres dokumentu                                      |

# Najmniejsze uprawnienia

Zasada udostępniania tylko operacji i danych niezbędnych do danego zadania.

## Dokumentacja obsługi

Wyszukiwanie i odczyt: `search_documents` oraz `read_document` z `workshop-docs`.

## Strona kontaktowa

Edycja tylko treści objętej wymaganiem. Zapis w zewnętrznej dokumentacji pozostaje poza zakresem.

# Pochodzenie informacji

Źródło zewnętrzne zawiera informacje poza repozytorium aplikacji.

| Element          | Znaczenie przy informacji o kontakcie                     |
|------------------|-----------------------------------------------------------|
| Tożsamość        | Identyfikator dokumentu i odpowiedzialny zespół obsługi   |
| Wersja (rewizja) | Konkretna odczytana publikacja dokumentu                  |
| Data             | Termin obowiązywania dokumentu, oddzielnie moment odczytu |
| Zakres           | Kanał wsparcia, dni obsługi i strefa czasowa              |

Brak wersji lub daty pozostaje jawną luką.

# Źródło prowadzi do sprawdzanej zmiany

| Etap        | Praca nad kontaktem                     | Podstawa kontroli                          |
|-------------|-----------------------------------------|--------------------------------------------|
| Informacja  | Reguła kontaktu z odczytanego fragmentu | Dokument i rewizja                         |
| Decyzja     | Wybór treści objętej wymaganiem         | Zakres dokumentu i zgłoszenia              |
| Mała zmiana | Prezentacja adresu i godzin obsługi     | Diff ograniczony do uzgodnionego miejsca   |
| Kontrola    | Zgodność tekstu i celu linku ze źródłem | Obserwacja strony wobec odczytanej rewizji |

# Brak źródła i sprzeczne dokumenty

| Sytuacja              | Decyzja                                                    | Podstawa                                              |
|-----------------------|------------------------------------------------------------|-------------------------------------------------------|
| Niedostępny odczyt    | Diagnoza połączenia i zgody narzędzia, potem ponowna próba | Ślad błędu albo rzeczywisty wynik                     |
| Różne godziny obsługi | Porównanie zakresu i daty obowiązywania                    | Nierozstrzygnięty konflikt wymaga decyzji właściciela |
| Wyciąg zastępczy      | Jawne wskazanie kopii i jej wersji                         | Użycie kopii nie dowodzi wywołania MCP                |

# Polecenie odczytu z dowodem

Cel: ustal <informację> dla <obszaru produktu>.

Serwer: workshop-docs.

Narzędzia: search\_documents i read\_document, tylko odczyt.

Wyszukaj dokument i odczytaj treść.

Ustal źródło, rewizję, datę i zakres obowiązywania.

Zwróć ślad: narzędzie, parametry, wynik, moment odczytu  
oraz fragment uzasadniający informację.

Oddziel potwierdzone fakty, luki i konflikty.

Zaproponuj decyzję i kontrolę zgodności ze źródłem.

Brak dostępu lub nierozstrzygnięty konflikt:

zwróć problem i podstawę, bez zgadywania.

Nie zmieniaj kodu ani dokumentacji.

# Handoff zadania

Przekazanie odtwarzalnego stanu pracy przez notatkę i aktualne źródła: celu, wykonanej części, dowodów oraz następnego kroku.

## Zastosowanie

Nowa sesja lub inna osoba przejmuje poprawkę komunikatu odmowy płatności w ShopEasy.

## Kryterium jakości

Potrzebne informacje pozwalają ustalić, od czego zaczyna się kolejny mały krok i jak zostanie sprawdzony.

# Kontekst służy następnemu krokowi

|                                 |                                                     |
|---------------------------------|-----------------------------------------------------|
| Materiał                        | Decyzja dla poprawki płatności                      |
| Aktualne zgłoszenie             | Pozostaje: cel i oczekiwane zachowanie              |
| Potrzebne fragmenty kodu i diff | Pozostają po kontroli aktualności                   |
| Dowody kontroli                 | Pozostają ze wskazaniem stanu repo, którego dotyczą |
| Pełny transkrypt                | Pominięty: powtórzenia i odrzucone pomysły          |
| Nieaktualny opis zachowania     | Pominięty po wskazaniu aktualnego źródła            |

# Notatka przekazania

Cel: poprawka komunikatu odmowy płatności w ShopEasy.

Stan: <wykonana część i dokładny punkt przerwania>.

Ustalenia: <potwierdzone fakty z odwołaniami>.

Założenia: <przyjęte reguły czekające na potwierdzenie>.

Dowody: <wykonane kontrole, wyniki i stan repo>.

Luki: <brakujące informacje lub decyzje>.

Następny krok: <jedno działanie, jego cel i kontrola>.

# Aktualne repo wyznacza punkt wznowienia

|                         |                                                    |
|-------------------------|----------------------------------------------------|
| Wskazanie w notatce     | Kontrola w nowej sesji                             |
| Stan poprawki płatności | Aktualny diff i zawartość potrzebnych źródeł       |
| Ustalenie o komunikacie | Wymaganie i dowód przypisany do tej wersji         |
| Następny mały krok      | Dostępne wejście oraz nierozstrzygnięte zależności |
| Wynik tego kroku        | Oczekiwane zachowanie, test i diff                 |

Rozbieżność z notatką wymaga aktualizacji planu przed dalszą zmianą.

# Reguła, procedura i stan mają właściwe miejsce

|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| Informacja z pracy nad płatnością | Miejsce aktualizacji                                     |
| Powtarzalna reguła projektu       | Właściwa instrukcja, np. wymóg dowodu dla opisu zmiany   |
| Sprawdzony sposób kontroli        | Istniejący Skill, np. powiązanie wymagania, testu i diff |
| Stan tej poprawki                 | Handoff: miejsce przerwania, wykonane kontrole i luki    |

Nowa sesja korzysta z potrzebnej instrukcji lub procedury przez odwołanie do aktualnego źródła i odczyt jego treści.

# Pomiar dotyczy konkretnej próby

Przykład zapisu dla wznowienia w VS Code/Copilot bez zebranego raportu zużycia.

| Miara              | Zapis obserwacji                                            |
|--------------------|-------------------------------------------------------------|
| Tokeny i cena      | Brak danych                                                 |
| Czas               | Od rozpoczęcia kroku do kontroli wyniku                     |
| Wywołania narzędzi | Zarejestrowane operacje, z jawnymi brakami śladu subagentów |
| Poprawki           | Zmiany po wykryciu niezgodności z wymaganiem                |

Czas nie mierzy tokenów. Porównanie wymaga tego samego kroku, stanu repo, modelu i kryterium jakości.

# Jakość poprawki utrzymują małe etapy

|                             |                                                         |
|-----------------------------|---------------------------------------------------------|
| Moment pracy                | Zachowana podstawa jakości komunikatu płatności         |
| Ustalenie zakresu           | Cel poprawki i zachowania objęte kontrolą               |
| Zakończenie etapu           | Mały diff powiązany z wymaganiem oraz wynikiem kontroli |
| Zmiana źródła lub założenia | Wskazanie ustaleń wymagających ponownego sprawdzenia    |
| Przerwanie pracy            | Aktualny stan, luki i następny możliwy krok             |
| Przyjęcie wyniku            | Opis zgodny z dowodami i jawnymi ograniczeniami         |

# PR/MR przedstawia propozycję zmiany

Pull Request / Merge Request to propozycja włączenia zmiany kodu, z miejscem na opis i review przed przyjęciem.

|                |                                                                       |
|----------------|-----------------------------------------------------------------------|
| Fragment opisu | Przykład dla darmowej dostawy w ShopEasy                              |
| Cel            | Zmiana warunku darmowej dostawy                                       |
| Przed          | Dla koszyka i metody opłata wynosiła wynik bazowy                     |
| Po             | Dla tego samego przypadku opłata wynosi potwierdzony wynik po zmianie |
| Zakres         | Miejsca w diff i wpływ na odbiorców kwoty                             |

# Status informacji określa rolę odbiorcy

| Zapis         | Treść przy zmianie dostawy                           | Rola odbiorcy                           |
|---------------|------------------------------------------------------|-----------------------------------------|
| Fakt          | Wynik przypadku na wersji, z odnośnikiem do kontroli | Reviewer weryfikuje twierdzenie         |
| Decyzja       | Przyjęta reguła lub kompromis, z uzasadnieniem       | Właściciel wymagania potwierdza zakres  |
| Założenie     | Niepotwierdzona interpretacja zachowania dostawy     | Wskazana osoba potwierdza albo odrzuca  |
| Otwarty punkt | Brak odpowiedzi o wariancie koszyka                  | Właściciel punktu ma następne działanie |

# Wynik kontroli ma wersję i zakres

Zapis wykonania obejmuje przypadek, rzeczywisty wynik i odnośnik do dowodu.

| Kontrola dostawy          | Informacje zachowane w opisie                                              |
|---------------------------|----------------------------------------------------------------------------|
| Wykonana przez autora     | Źródło pakietu, data, wersja, przypadek, zakres i wynik                    |
| Własne wykonanie          | Bieżąca wersja kopii, przypadek, zakres, wynik i dowód z tej próby         |
| Planowana lub niewykonana | Cel kontroli wariantu, przyczyna braku wykonania, bez deklarowanego wyniku |

# Ograniczenie dowodu wpływa na decyzję

Opis zachowuje luki, które zmieniają ocenę gotowości. Przykłady dotyczą sytuacji, gdy wskazane ograniczenie występuje.

| Sytuacja przy dostawie              | Granica wniosku                      | Potrzebna decyzja                           |
|-------------------------------------|--------------------------------------|---------------------------------------------|
| Kontrola obejmuje jedną metodę      | Brak potwierdzenia pozostałych metod | Uzupełnienie wymaganego zakresu             |
| Dowód dotyczy innej wersji niż diff | Wynik nie potwierdza bieżącej zmiany | Kontrola opisywanej wersji                  |
| Założenie wpływa na regułę          | Możliwa niezgodność z wymaganiem     | Rozstrzygnięcie przez właściciela wymagania |

# Opis PR/MR

Szablon wiąże twierdzenia z aktualnym diff i dowodami.  
Szczegóły dostawy pozostają parametrami konkretnej zmiany.

Wejście: <kontrakt>, <diff>, <dowody>  
Cel: <potrzeba, zachowanie przed/po>  
Zakres: <pliki i istotny wpływ>  
Decyzje: <co przyjęto i dlaczego>  
Założenia i pytania: <status, właściciel>  
Wykonane kontrole: <wersja, przypadek,  
zakres, wynik, odnośnik do dowodu>  
Planowane kontrole: <cel, brak wykonania>  
Ryzyka i ograniczenia: <skutek dla review>  
Dokumentacja: <sekcja, źródło w kodzie>

# Referencja techniczna wynika z aktualnego kodu

Dokumentacja techniczna opisuje implementację i użycie. Aktualizacja jednej sekcji dostawy wymaga porównania z bieżącym kodem.

| Zdanie referencji | Potwierdzenie                       | Efekt aktualizacji                    |
|-------------------|-------------------------------------|---------------------------------------|
| Reguła dostawy    | Aktualny kontrakt i implementacja   | Opis zgodny z zachowaniem             |
| Plik lub symbol   | Odczyt rzeczywistego miejsca logiki | Odkaz umożliwiający weryfikację       |
| Zakres kontroli   | Treść testu i wykonany wynik        | Informacja o sprawdzonych przypadkach |

# Pierwsza próba metody we własnym projekcie

| Element planu     | Konkretne ustalenie                                                 |
|-------------------|---------------------------------------------------------------------|
| Sytuacja          | Najbliższe małe zgłoszenie: reguła w projekcie                      |
| Źródła            | Wymaganie, kod, standardy zespołu, test                             |
| Granice           | Pliki i narzędzia, jedna reguła, bez publikacji                     |
| Pierwsza kontrola | Przypadek odtworzony na bazowej wersji, przed zmianą                |
| Ocena             | Zgodność przypadku z kontraktem i liczba poprawek do przyjęcia diff |
| Utrwalenie        | Procedura zachowana i użyta na drugim przypadku                     |